High-Performance Visual Telemetry: An Analysis of JPEG Transmission via ESP-NOW in Distributed Embedded Systems

The pursuit of efficient, low-latency image transmission in resource-constrained environments has led to the emergence of specialized communication protocols that bypass the traditional overhead of the Transmission Control Protocol/Internet Protocol (TCP/IP) stack. In the domain of Espressif-based microcontrollers, the ESP-NOW protocol serves as a connectionless, peer-to-peer communication mechanism that facilitates direct interaction between devices at the data-link layer.[1] For developers utilizing the ESP32-CAM module, the integration of ESP-NOW for JPEG transmission represents a critical design choice aimed at maximizing battery life, extending range, and reducing the temporal gap between image capture and remote display.[4] This report provides an exhaustive technical analysis of the mechanisms, limitations, and optimization strategies associated with sending visual data over ESP-NOW, synthesizing findings from official documentation, peer-reviewed benchmarks, and extensive developer forum discussions.

Foundations of the ESP-NOW Communication Protocol

The architectural philosophy of ESP-NOW is rooted in the simplification of the networking stack to achieve near-instantaneous data transfer. Developed by Espressif Systems, the protocol operates as a proprietary extension of the IEEE 802.11 standard, specifically utilizing vendor-specific action frames to encapsulate application data.[7] Unlike standard Wi-Fi station (STA) or Access Point (AP) modes, which require time-intensive handshaking, authentication, and DHCP address assignment, ESP-NOW enables devices to communicate immediately upon power-up, provided they possess the Media Access Control (MAC) address of the peer.[3]

Technical Specifications and Versioning

The evolution of ESP-NOW has seen significant improvements in its data-handling capacity. The original iteration, referred to as v1.0, is characterized by a 250-byte maximum payload per packet.[1] This constraint is a direct consequence of the IEEE 802.11 management frame structure, where the length field for vendor-specific information elements is restricted to 1 byte, limiting the total element size to 255 bytes inclusive of headers.[7] In contrast, ESP-NOW v2.0, supported in modern versions of the ESP-IDF, expands this envelope to a maximum

packet length of 1470 bytes, designated as ESP_NOW_MAX_DATA_LEN_V2.[8]

Feature	ESP-NOW v1.0	ESP-NOW v2.0
Maximum Payload	250 Bytes [1]	1470 Bytes [8]
PHY Layer	802.11 b/g/n [11]	802.11 b/g/n/ax (on newer SoCs) [12]
Default Bitrate	1 Mbps [7]	1 Mbps (Configurable) [7]
Framing Method	Vendor-specific IE [7]	Action Frame Encapsulation [8]
Pairing Limit	20 Peers [1]	20 Peers (Adjustable up to 17 encrypted) [7]

The implications of these payload limits are profound for image transmission. A standard VGA ($640 \times$) JPEG image, which may range from 15 KB to 30 KB depending on quality settings, requires approximately 60 to 120 packets under v1.0 protocols but only 10 to 20 packets under v2.0.[10] This reduction in packet fragmentation decreases the probability of bitstream corruption and minimizes the computational overhead associated with packet header processing and reassembly.[10]

Operational Modes and Network Topologies

ESP-NOW supports several networking configurations that cater to different telemetry requirements. The simplest configuration is a one-way, point-to-point link where an "Initiator" (the ESP32-CAM) transmits data to a "Responder" (a receiving ESP32 or ESP8266).[9] For more complex deployments, the protocol allows for broadcast mode using the MAC address FF:FF:FF:FF:FF:FF, where all devices on the same Wi-Fi channel receive the transmission without providing a hardware-level acknowledgement (ACK).[17] While broadcast mode can increase throughput by eliminating ACK wait times, it significantly compromises reliability in high-interference environments, as the sender has no mechanism to detect lost packets.[3]

Hardware Synergy: The ESP32-CAM Module

The successful transmission of JPEG data begins with the hardware capabilities of the ESP32-CAM board. This module typically integrates an ESP32-S SoC with an OV2640 image sensor, supported by external Pseudo-static RAM (PSRAM) to bridge the gap between sensor

throughput and microcontroller memory limitations.[11]

Sensor Integration and JPEG Encoding

The OV2640 sensor is a 2-megapixel module that provides built-in hardware JPEG encoding.[6] This feature is vital for ESP-NOW applications because it offloads the computationally expensive discrete cosine transform (DCT) and Huffman coding tasks from the ESP32's dual-core Xtensa processor.[6] The sensor interfaces with the SoC via a Digital Video Port (DVP), which transmits the compressed bitstream in a parallel format.[6]

The resolution of the capture directly influences the feasibility of transmission over a connectionless link. While the sensor supports resolutions up to UXGA ($1600 \times$), the resulting file sizes (often exceeding 100 KB) can lead to significant latency and increased packet loss probability when transmitted via ESP-NOW.[6] Consequently, forum discussions and research benchmarks often highlight VGA ($640 \times$) or QVGA ($320 \times$) as the "sweet spot" for balancing visual clarity with transmission reliability.[11]

Memory Constraints and PSRAM Utilization

A critical limitation of the base ESP32 chip is its internal SRAM, which is insufficient for buffering high-resolution image frames while simultaneously managing the Wi-Fi stack and application logic.[11] The inclusion of 4MB of external PSRAM on most ESP32-CAM modules allows the camera driver to allocate large frame buffers.[11] The driver uses Direct Memory Access (DMA) to move the JPEG data from the DVP directly into PSRAM, ensuring that the CPU is available to initiate the ESP-NOW transmission sequence as soon as the frame is complete.[6]

Resolution Name	Dimensions	Typical JPEG Size (Quality 12)	Memory Requirement
QQVGA	$160 \times$	2.9 KB	Low (SRAM sufficient) [11]
QVGA	$320 \times$	5.9 KB	Moderate (SRAM/PSRAM) [11]
VGA	$640 \times$	15.5 KB	High (PSRAM recommended) [11]
SVGA	$800 \times$	26.9 KB	PSRAM mandatory [11]

UXGA	$1600 \times$	108.4 KB	PSRAM mandatory [11]

Engineering the JPEG Transmission Pipeline

Because ESP-NOW is not inherently designed for file transfer, transmitting a JPEG requires a custom application-layer protocol to handle fragmentation, transmission, and reassembly.[10] This pipeline must be engineered to handle the specificities of the JPEG format, where the loss of a single byte can desynchronize the entropy-coded bitstream and render the entire image unviewable.[10]

Fragmentation and Packet Header Strategy

The sender must divide the JPEG buffer stored in PSRAM into chunks that fit within the ESP_NOW_MAX_DATA_LEN.[10] Effective reassembly at the receiver necessitates the inclusion of a metadata header in every packet.[10] A common header structure includes a 1-byte frame identifier (to distinguish between different images), a 2-byte sequence number (to ensure packets are reassembled in the correct order), and a 2-byte total packet count.[10]

When utilizing ESP-NOW v1.0, a 250-byte packet with a 5-byte header leaves 245 bytes for JPEG data.[10] For a 15 KB image, this results in approximately 62 packets. The transmission logic utilizes pointer arithmetic to iterate through the frame buffer: $BufferOffset = SequenceNumber \times MaxPayloadSize$.[10]

Receiver-Side Reassembly and Error Handling

The receiver must pre-allocate a reassembly buffer in its own PSRAM or SRAM.[22] As packets arrive through the esp_now_register_recv_cb callback, the application parses the header, extracts the sequence number, and copies the payload into the correct offset of the reassembly buffer.[10]

A significant challenge identified in developer forums is the lack of a built-in retry mechanism for lost packets in ESP-NOW.[3] While unicast mode provides a hardware ACK, this only informs the sender that the packet reached the radio; it does not guarantee that the application layer processed it.[18] To ensure "no missing bits," advanced implementations incorporate a Negative Acknowledgement (NACK) system, where the receiver identifies gaps in the sequence numbers and requests specific retransmissions from the sender.[21]

Impact of JPEG Quality and Compression

The jpeg_quality parameter, ranging from 0 (highest quality) to 63 (lowest quality), is a primary lever for controlling transmission performance.[11] Lower quality values result in larger files but

higher fidelity, whereas higher values (10–20 is the practical range) significantly reduce the number of ESP-NOW packets required per frame.[11] Reducing the packet count not only decreases latency but also improves the overall success rate of image transfer, as fewer packets represent fewer opportunities for interference-induced corruption.[11]

Performance Benchmarking and Throughput Analysis

Throughput in ESP-NOW is influenced by the physical (PHY) rate, the Wi-Fi channel environment, and the protocol overhead.[7] Test results from Espressif indicate that the default 1 Mbps PHY rate typically yields an effective application-layer bit rate of approximately 214 Kbps in open environments and 555 Kbps in shielding boxes.[7]

PHY Rate Optimization

Developers can increase throughput by configuring the PHY rate using the esp_wifi_config_espnow_rate() function.[7] Setting the rate to higher Modulation and Coding Scheme (MCS) values can significantly boost speed, but this comes with a trade-off in range and signal robustness.[7]

PHY Rate Constant	Theoretical Max	Observed Throughput (Indoor)	Reliability Cost
WIFI_PHY_RATE_1M_L	1 Mbps	~250 Kbps	Very High Reliability [7]
WIFI_PHY_RATE_11M_L	11 Mbps	~800 Kbps	High Reliability [18]
WIFI_PHY_RATE_24M	24 Mbps	~550 KB/s	Moderate Loss [18]
WIFI_PHY_RATE_54M	54 Mbps	Minimal	Extreme Packet Loss [18]

Research indicates that a bit rate of $549,250$ bytes/s can be achieved at WIFI_PHY_RATE_24M, which would theoretically allow a 25 KB VGA image to be transmitted in approximately 45 ms.[18] However, at the highest rates (54 Mbps), nearly all packets are typically

lost in non-ideal conditions, making the mid-range MCS rates more suitable for stable video or image streaming.[18]

Latency and Frame Rate Benchmarks

The latency of an ESP-NOW image link is the sum of capture time, encoding time, fragmentation time, and total airtime for all packets.[11] For a QVGA stream at quality 12, the ESP32-CAM can achieve 20–25 FPS over standard Wi-Fi, but the fragmentation overhead of ESP-NOW v1.0 typically limits this to 10–15 FPS in real-world P2P scenarios.[11] Using the ESPNowCam library or raw 802.11tx frames can push these limits further, with some reports showing 30 FPS at lower resolutions.[14]

Signal Integrity and Range Optimization

ESP-NOW is frequently chosen for its superior range characteristics compared to traditional Wi-Fi. In open-field conditions using the on-board PCB antenna, stable communication is typically achieved at distances of 165 to 220 meters.[3]

Hardware Modifications for Extended Range

The range of the ESP32-CAM can be dramatically extended through antenna engineering. Most modules include an IPEX (U.FL) connector for an external antenna, but activating it requires moving a zero-ohm resistor on the board to disconnect the internal PCB trace.[5] With a high-gain external antenna, developers have reported successful image transmission at distances exceeding 400 to 650 meters in line-of-sight environments.[5]

Long Range (LR) Mode and Environmental Factors

Espressif's Long Range (LR) mode is a specialized PHY configuration that can extend the signal beyond 900 meters.[5] However, LR mode reduces the available data rate to 512 Kbps or 256 Kbps, which necessitates a significant reduction in JPEG resolution or an increase in the time required to send a single frame.[5] Furthermore, signal integrity is heavily influenced by polarization; ensuring that both the transmitter and receiver antennas are vertically oriented minimizes signal loss due to polarization mismatch.[26]

Network Coexistence and Gateway Architectures

A common requirement is for a "gateway" node to receive ESP-NOW data from several remote cameras and forward it to the internet via standard Wi-Fi.[4] This necessitates the simultaneous operation of the ESP-NOW protocol and the Wi-Fi station (STA) mode.[12]

Channel Synchronization Constraints

The ESP32 radio can only operate on one Wi-Fi channel at a time.[7] Therefore, the ESP-NOW communication must occur on the same channel that the gateway node uses to connect to its local Access Point.[7] This introduces a significant complication: if the Wi-Fi router dynamically changes channels, the ESP-NOW nodes will lose connection.[12]

To mitigate this, developers implement channel-scanning logic in the remote nodes. Upon a transmission failure, the remote camera cycles through all 11–13 Wi-Fi channels, broadcasting a "hello" or "probe" frame until it receives a response from the gateway, thereby synchronizing with the new channel.[12]

Bridging to the Cloud

Once the JPEG is reassembled on the gateway, it can be uploaded to a server using HTTP POST or MQTT.[4] Projects like ESPNowCam and Radin-Target-Camera demonstrate this architecture, where the reassembled image is stored in the gateway's SPIFFS or LittleFS filesystem before being served to a web browser or pushed to a remote PHP server.[4] This dual-protocol operation requires careful management of radio time-slicing to prevent the Wi-Fi stack from blocking ESP-NOW reception.[12]

Security and Robustness in Peer-to-Peer Links

Image data, especially in surveillance or remote monitoring contexts, may require protection against eavesdropping or tampering. ESP-NOW supports hardware-accelerated encryption using AES-128 and the CCMP protocol.[1]

Encryption and Pairing Limits

The protocol supports a limited number of encrypted peers: typically 10 in Station mode and 6 in SoftAP mode, though this can be increased to 17 in the configuration menu of modern SDKs.[1] Encryption requires the pre-sharing of a Primary Master Key (PMK) and a Local Master Key (LMK) between the camera and the receiver.[1] It should be noted that broadcast packets cannot be encrypted, meaning that any image sent to the FF:FF:FF:FF:FF:FF address is visible to any ESP32 in sniffer mode within range.[7]

Resilience to Replay Attacks

Recent analysis of the ESP-NOW protocol identified a vulnerability to replay attacks, where an attacker could capture and re-transmit action frames.[33] Espressif addressed this by incorporating a "magic" value—a 16-bit random number—into every packet.[33] The receiving node caches these values in g_msg_magic_cache and rejects any incoming packet with a magic value that has recently been seen, effectively nullifying replay attempts.[33] For visual data,

this protection ensures that an attacker cannot inject "old" image packets into a live stream to deceive the receiver.

Advanced Techniques: Raw 802.11tx and Sniffer Mode

For high-performance applications that exceed the capabilities of the standard ESP-NOW stack, developers utilize the esp_wifi_80211_tx() API.[7] This method allows for the transmission of raw Wi-Fi frames, bypassing the ESP-NOW vendor-specific action frame structure entirely.[7]

Performance Gains and Implementation Complexity

Using the 80211tx() primitive can result in a performance improvement of approximately 40% compared to standard ESP-NOW P2P mode.[22] This is achieved by reducing the internal processing time of the Espressif stack and allowing for more efficient packet scheduling.[22]

However, the implementation complexity is significantly higher. The sender must manually construct the 802.11 MAC headers, and the receiver must be placed in promiscuous/sniffer mode to capture the frames.[7] In sniffer mode, the standard TCP/IP stack is disabled, meaning the receiving device cannot simultaneously connect to a Wi-Fi router unless it uses a dual-radio setup or complex time-multiplexing.[7]

Transmission Method	Throughput	Complexity	Use Case
Standard ESP-NOW	Moderate	Low	General IoT data, sensor readings [3]
ESP-NOW v2.0	High	Moderate	Larger data blocks, file transfer [8]
802.11tx Raw	Very High	High	High-speed video streaming, high-res images [22]

Troubleshooting Common Forum-Identified Issues

Extensive discussion in developer communities like the Arduino Forum and Reddit highlights several recurring challenges when implementing ESP-NOW image transmission.

The Brownout Problem

A frequent cause of failure is the "brownout" error, where the ESP32 resets during a

transmission burst.[15] Image capture and Wi-Fi transmission are the two most power-intensive operations for the ESP32. When the radio initiates a transmission burst for 60+ JPEG packets, the current draw can exceed 240 mA.[35] If the power supply is inadequate or the voltage regulator on the ESP32-CAM (often a low-dropout 3.3V LDO) is weak, the voltage drops, triggering the hardware reset circuit.[15] The recommended solution is to use a high-quality 5V power supply and add a large electrolytic capacitor (e.g., 100μF to 470μF) across the 5V and GND pins of the module.[19]

Clock Jitter and Network Stability

On devices using both Ethernet and ESP-NOW, users have reported ICMP packet loss and UDP instability.[32] This is often traced to the internal clock source; when the Wi-Fi radio is active, it can interfere with the RMII clock stability required for Ethernet.[32] The workaround suggested by Espressif is to use an external crystal for the Ethernet clock or to ensure that ESP-NOW transmission intervals are sufficiently spaced to allow the Ethernet controller to maintain its link.[32]

Packet Loss in Congested 2.4 GHz Environments

In environments with many competing Wi-Fi networks, the probability of packet collision increases significantly.[3] Because ESP-NOW uses a fixed channel, it cannot perform the "Listen Before Talk" (CSMA/CA) mechanism as robustly as standard Wi-Fi station mode.[3] For large data transfers like JPEG images, this results in "striped" or "corrupted" images where sections of the JPEG are missing.[21] Forum experts suggest implementing a cyclic redundancy check (CRC) at the end of the reassembled image to verify integrity before attempting to decode or display the file.[10]

Software Frameworks and Libraries

To avoid "reinventing the wheel," several open-source libraries have been developed specifically for ESP-NOW data streaming.

1. **ESPNowCam:** This library is a direct video or data streamer designed for ESP32 devices.[22] it supports various transmission modes and has been tested with popular cameras like the AI-Thinker and M5UnitCam.[22] It simplifies the chunking process and provides a callback-based architecture for handling reassembled frames.[22]
2. **Radin-Target-Camera:** This project focuses on long-range image transmission using ESP-NOW to relay photos to a gateway node.[27] It specifically handles the logic for storing images in SPIFFS and serving them via a web interface, making it an excellent reference for coexistence strategies.[27]
3. **SerialTransfer:** While primarily intended for serial communication, the SerialTransfer.h library can be adapted to packetize any data buffer for ESP-NOW.[15] It provides a robust

framing system that includes CRC and packet sequence tracking, which are essential for reliable JPEG transfer.[15]

Synthesis and Recommendations for Implementation

The research indicates that while ESP-NOW is "not for huge data transfer" by design, it can be engineered into a highly effective medium for JPEG telemetry.[35] The primary advantage of this approach is the speed of activation and the efficiency of transmission, which far exceeds HTTP-based methods for battery-powered remote cameras.[5]

For optimal results, the following technical strategies should be adopted:

- **Protocol Selection:** Use ESP-NOW v2.0 if the SDK version allows, as the 1470-byte packet limit reduces the number of transmission cycles and minimizes fragmentation errors.[8]
- **Resolution and Quality:** Standardize on QVGA or VGA resolution with a quality factor between 10 and 15 to keep packet counts manageable while maintaining acceptable visual fidelity.[11]
- **Power Management:** Ensure a dedicated 5V power rail with external capacitance to prevent radio-induced brownouts.[19]
- **Reliability Logic:** Implement a custom NACK or selective-retransmission system at the application layer to handle the inevitable packet loss in the 2.4 GHz band.[3]
- **Channel Management:** If a gateway node is used, implement a robust channel-scanning routine to keep the remote cameras synchronized with the gateway's Wi-Fi channel.[12]

By combining these strategies, the ESP32-CAM becomes a potent tool for distributed sensing, capable of delivering real-time visual data over distances and with a power profile that standard Wi-Fi cannot match. The integration of hardware-level JPEG encoding with the lightweight ESP-NOW protocol represents a sophisticated optimization of the ESP32 platform, suitable for applications ranging from agricultural monitoring to industrial safety systems.[4]

Works cited

1. Esp-Now User Guide en | PDF | Network Packet | Transmission Control Protocol - Scribd, accessed March 18, 2026, https://www.scribd.com/document/326479501/Esp-now-User-Guide-En
2. ESP32 - Part 09 - ESP-NOW, accessed March 18, 2026, https://wp.engrie.be/wp-content/uploads/2025/03/ESP32_-_Part_09_-_ESP-NOW.pdf
3. Device to Device Communication with ESP-NOW - Arduino Documentation, accessed March 18, 2026, https://docs.arduino.cc/tutorials/nano-esp32/esp-now
4. dualvim/ESP32CAM_ESPNOW_Sensors_WebServer: ESP32 and ESP32-CAM: Web Server and ESP-NOW protocol - GitHub, accessed March 18, 2026, https://github.com/dualvim/ESP32CAM_ESPNOW_Sensors_WebServer
5. ESP-NOW Tutorial for Long-Range Communication: Building Robust Wireles,

accessed March 18, 2026, https://thinkrobotics.com/blogs/learn/esp-now-tutorial-for-long-range-communication-building-robust-wireless-networks-in-2025
6. (PDF) A Benchmark Reference for ESP32-CAM Module - ResearchGate, accessed March 18, 2026, https://www.researchgate.net/publication/392314002_A_Benchmark_Reference_for_ESP32-CAM_Module
7. ESP-NOW - - — ESP-FAQ latest documentation, accessed March 18, 2026, https://docs.espressif.com/projects/esp-faq/en/latest/application-solution/esp-now.html
8. ESP-NOW - ESP32 - — ESP-IDF Programming Guide v5.5.3 ..., accessed March 18, 2026, https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
9. ESP NOW - Peer to Peer ESP32 Communications - DroneBot Workshop, accessed March 18, 2026, https://dronebotworkshop.com/esp-now/
10. c++ - Using esp_now for sending more than packet - Stack Overflow, accessed March 18, 2026, https://stackoverflow.com/questions/73654827/using-esp-now-for-sending-more-than-packet
11. ESP32 Camera Streaming: MJPEG Video to Browser Tutorial - Zbotic, accessed March 18, 2026, https://zbotic.in/esp32-camera-streaming-mjpeg-video-to-browser-tutorial/
12. ESPNow and WiFi at the Same Time – Yes, It's Possible (and Practical) : r/esp32 - Reddit, accessed March 18, 2026, https://www.reddit.com/r/esp32/comments/1mowecs/espnow_and_wifi_at_the_same_time_yes_its_possible/
13. A Benchmark Reference for ESP32-CAM Module - arXiv.org, accessed March 18, 2026, https://arxiv.org/html/2505.24081v1
14. ESP32-CAM Video Streaming – What's the Max FPS You've Managed? - Reddit, accessed March 18, 2026, https://www.reddit.com/r/esp32/comments/1k2vobi/esp32cam_video_streaming_whats_the_max_fps_youve/
15. Transfer pictures from ESP32-CAM to ESP32 via serial - Programming - Arduino Forum, accessed March 18, 2026, https://forum.arduino.cc/t/transfer-pictures-from-esp32-cam-to-esp32-via-serial/647488
16. ESP32 ESP-NOW Getting Started Tutorial with Arduino IDE - Microcontrollers Lab, accessed March 18, 2026, https://microcontrollerslab.com/esp32-esp-now-tutorial-arduino-ide/
17. ESP32 With ESP-Now Protocol : 16 Steps - Instructables, accessed March 18, 2026, https://www.instructables.com/ESP32-With-ESP-Now-Protocol/
18. ESP-NOW Data Transfer | PS9888 2020 - Smart Dataloggers, accessed March 18, 2026, https://blogs.ntu.edu.sg/ps9888-2020-g16/2020/07/11/esp-now-data-transfer/

19. ESP32-Cam - Complete Guide - DIY Engineers, accessed March 18, 2026, https://www.diyengineers.com/2023/04/13/esp32-cam-complete-guide/
20. GitHub - s60sc/ESP32-CAM_MJPEG2SD: ESP32 Camera motion capture application to record JPEGs to SD card as AVI files and stream to browser as MJPEG. If a microphone is installed then a WAV file is also created. Files can be uploaded via FTP or downloaded to browser., accessed March 18, 2026, https://github.com/s60sc/ESP32-CAM_MJPEG2SD
21. ESP32-CAM Image sending via WI-FI Mesh - Arduino Forum, accessed March 18, 2026, https://forum.arduino.cc/t/esp32-cam-image-sending-via-wi-fi-mesh/1276472
22. ESPNowCam, a straightforward video streamer for popular ESP32Cam models, leveraging the ESPNow and WiFi-raw protocols. No need for IPs, routers, or credentials-keeping it simple! - GitHub, accessed March 18, 2026, https://github.com/hpsaturn/ESPNowCam
23. How To Get Started With ESP-NOW - DigiKey, accessed March 18, 2026, https://www.digikey.com/en/maker/tutorials/2024/how-to-get-started-with-esp-now
24. Improving Esp32-CAM Image Quality : 7 Steps - Instructables, accessed March 18, 2026, https://www.instructables.com/Improving-Esp32-CAM-Image-Quality/
25. Wi-Fi Driver - ESP32 - — ESP-IDF Programming Guide v5.2 documentation, accessed March 18, 2026, https://docs.espressif.com/projects/esp-idf/en/v5.2/esp32/api-guides/wifi.html
26. ESP-NOW vs WiFi range : r/esp32 - Reddit, accessed March 18, 2026, https://www.reddit.com/r/esp32/comments/1dmwz4p/espnow_vs_wifi_range/
27. BL08FR/Radin-Target-Camera-Long-range-Esp-Now-Camera - GitHub, accessed March 18, 2026, https://github.com/BL08FR/Radin-Target-Camera-Long-range-Esp-Now-Camera
28. wifi and esp-now.... again! : r/esp32 - Reddit, accessed March 18, 2026, https://www.reddit.com/r/esp32/comments/st4uno/wifi_and_espnow_again/
29. ESP32-CAM HTTP Post Images/Photos to Server - Random Nerd Tutorials, accessed March 18, 2026, https://randomnerdtutorials.com/esp32-cam-post-image-photo-server/
30. ESP32-CAM HTTP POST Photos to Local or Cloud Server using PHP and Arduino IDE, accessed March 18, 2026, https://embedgyan.wordpress.com/2020/07/11/esp32-cam-http-post-photos-to-local-or-cloud-server-using-php-and-arduino-ide/
31. ESP32-CAM Take Photo and Display in Web Server - Random Nerd Tutorials, accessed March 18, 2026, https://randomnerdtutorials.com/esp32-cam-take-photo-display-web-server/
32. Ethernet packet loss with ESP-NOW (idf v4.4.6) - ESP32 Forum, accessed March 18, 2026, https://esp32.com/viewtopic.php?t=37757
33. Flaws in Espressif ESP-NOW Allow Attackers to Replay Communications - Nozomi Networks, accessed March 18, 2026, https://www.nozominetworks.com/blog/flaws-in-espressif-esp-now-allow-attackers-to-replay-communications

34. ESP32 Camera transmission and reception via ESPNow protocol - Proof of concept., accessed March 18, 2026, https://www.youtube.com/watch?v=zXIzP1TGIpA
35. Data transmission reliability over ESP-NOW protocol in indoor environment, accessed March 18, 2026, https://developer.espressif.com/blog/reliability-esp-now/
36. esp-idf/examples/wifi/espnow/main/espnow_example_main.c at master - GitHub, accessed March 18, 2026, https://github.com/espressif/esp-idf/blob/master/examples/wifi/espnow/main/espnow_example_main.c
37. esp32cam · GitHub Topics, accessed March 18, 2026, https://github.com/topics/esp32cam
38. I used ESP-NOW to create a networking system for sensors on my vegetable farm without using WiFi : r/esp32 - Reddit, accessed March 18, 2026, https://www.reddit.com/r/esp32/comments/o98x8p/i_used_espnow_to_create_a_networking_system_for/